

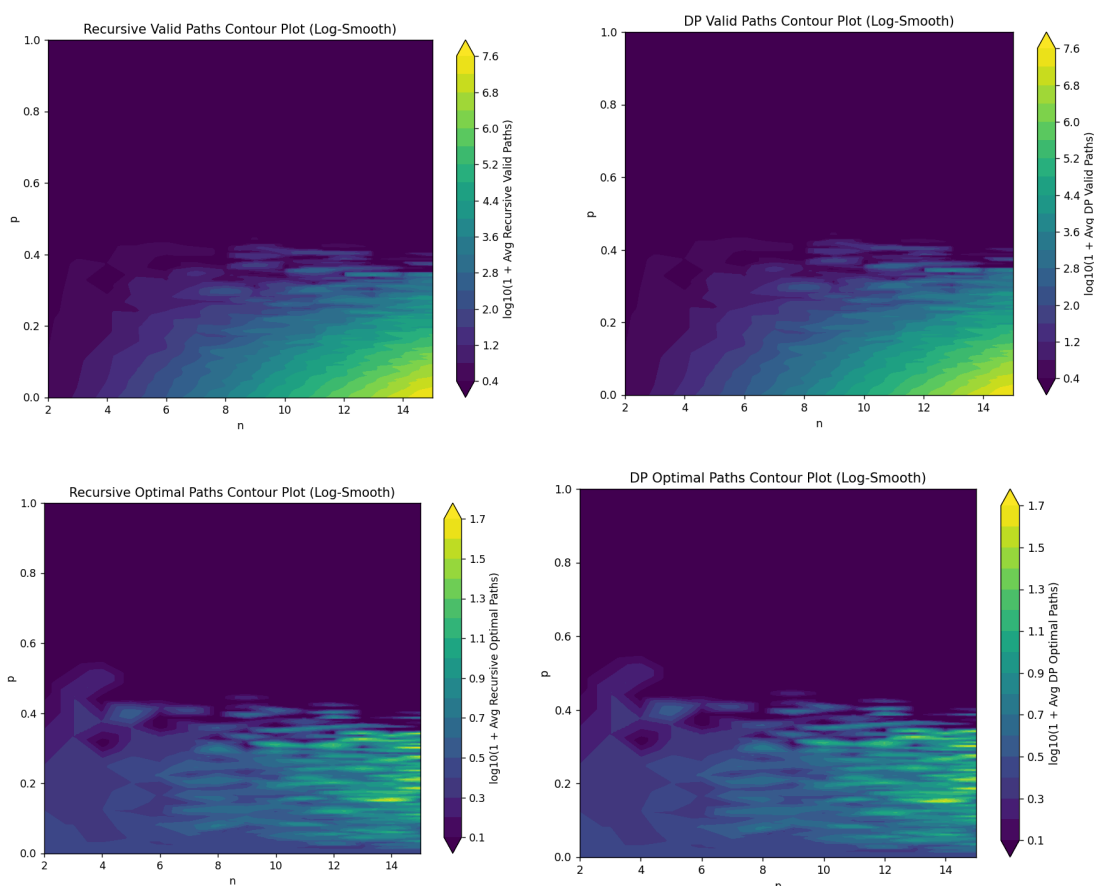
題目說明：

在現實情境中，如車輛在行駛地圖時，轉彎的次數會影響車輛到達終點的時間，轉彎數越多，車輛到達時間越晚。因此，我們將 "optimal path" 定義為轉彎數最少的路徑，並排除所有沒有有效路徑 (valid path) 的情況。

1. 實現運算式與程式碼，並輸入不同的 n 值運行以觀察結果：

請參閱 github [Ping3227/PathFinding \(github.com\)](https://github.com/Ping3227/PathFinding) 或附件程式

2. 繪製趨勢圖：

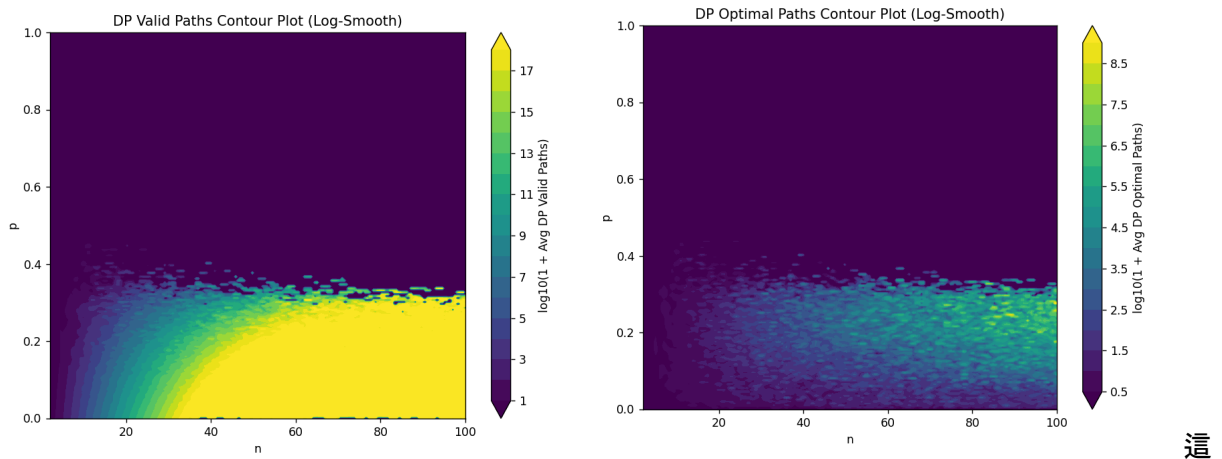


將隨著 n 增加時，最佳路徑數量的增長趨勢以 $\log(N+1)$ 形式繪製，並觀察 valid path 的變化趨勢。結果顯示，valid path 大約在密度從 0 到 0.4 左右時指數遞減，並且隨著 n 增加以指數速度增加。從公式解 $\frac{2n!}{n!n!}$ 可以看出 valid path 的成長趨勢。

同時，optimal path 的增長速度相對較慢，並且通常在障礙物密度達到 0.3 左右時才會達到高峰。

3. 演算法與實際應用中的重要性。:

演算法說明請參readme



個問題的核心在於 valid path 呈現指數級的增長，因此暴力窮舉的效率極差。在實際應用中，這對於隨機生成地圖的設計有重要意義。根據上述分析，可以設計合理的障礙物密度，確保遊戲中有更多的最佳解。現代許多遊戲在關卡設計中，會提供多個可行解，並且保證至少存在一個可行解以提升遊戲體驗。

4. 挑戰: 比較不同的演算法:

我們可以將此問題用動態規劃 (Dynamic Programming, DP) 解決。DP 可以將原本暴力法 $O(2^n)$ 的時間複雜度壓縮到 $O(n^2)$ ，但代價是增加 $O(n^2)$ 的空間使用。另外使用 DP 填表的方式，還具備了更好的在線更新能力 (online update)，當新增新的外部地圖時，能夠透過現有的表格用更快的速度更新結果，以遞迴的方式可能需要重新計算。

性能測試:

- 當 $n=2$ 時，遞迴法與 DP 的耗時幾乎相同。
- 當 $n=10$ 時，DP 的時間為 0ms，而遞迴法為 10ms。
- 當 $n=15$ 時，DP 仍為 0ms，而遞迴法需要 1370ms。
- 當 $n=1000$ 時，DP 只需要 428ms，然而遞迴法所花費的時間已經難以估計。

這證明了 DP 在面對大規模問題時比遞迴法更具效率，尤其在遊戲設計或地圖生成等實際應用中，能顯著提升運算性能。